Cybersecurity Project Research Roadmap

OpenSource...ersecurity PDF

Physics_In...l_Research PDF

SNR_Filter...l_Research PDF

I am working on a project as the lead developer in my company Nyther Research Pvt LTd.

I have to develop these tools or cybersecurity softwares such as 1) Physics Informed Intrsuion Detection & Response and 2) Side Channel Analysis usind Electromagnetic wave.

The above shared docs have links and some content about the sources where we can find to start our research or how to develop these projects.

Please read them and help me in my research acting as a senior researcher and help be build this project

As the senior researcher supporting your work at Nyther Research Pvt Ltd, I have synthesized a technical roadmap based on the provided documentation to help you develop your dual-track cybersecurity projects.

1. Physics-Informed Intrusion Detection & Response (PI-IDR)

Traditional IDS often fails to detect zero-day attacks or subtle physical-layer anomalies in Cyber-Physical Systems (CPS) . Physics-informed IDS addresses this by embedding physical governing equations (e.g., power flow laws, kinematics, Maxwell's equations) directly into the AI detection pipeline.

Core Mathematical Foundation

- **PINN Loss Framework**: You will use Physics-Informed Neural Networks (PINNs) where the training loss (L_{total}) enforces both data fidelity (L_{data}) and physical law (L_{pde}).
- **Anomaly Scoring**: At inference, an intrusion is flagged if the anomaly score $A(t)$, calculated as the normalized difference between observed and predicted physical states, exceeds a predefined Neyman-Pearson threshold (τ).

Recommended Open-Source Stack

- **DeepXDE**: The most widely adopted library for solving forward and inverse PDE/ODE problems. It is ideal for embedding physical residuals into neural network training to flag physically-inconsistent sensor readings.
- **NVIDIA Modulus**: Used for building physics-aware "digital twins" at scale, particularly suitable for SCADA and power grid security.
- **Stratosphere Linux IPS (Slips)**: A behavioral IDS that uses Kalman filter-based temporal models to track expected network state evolution, treating network flow as a physical state-space system.

---

2. Side-Channel Analysis (SCA) using Electromagnetic Waves

EM side-channel analysis monitors non-invasive radiation from embedded controllers to detect program control flow deviations.

Key Research Foundations

- **Control Flow Monitoring**: Research by Han et al. (2017) demonstrated that complex execution paths leave unique EM fingerprints that can be used for contactless monitoring.
- **Multi-Layer Detection**: Modern approaches integrate EM and power leakage with network traffic data for a three-tier leakage-to-alert pipeline, validated on ARM Cortex-M targets.

Signal-to-Noise Ratio (SNR) Optimization

To make your detection reliable, you must separate exploitable physical signals from background noise:

- **NICV (Normalized Inter-Class Variance)**: A foundational metric to detect relevant leakage points using only public data (plaintext/ciphertext).
- **TVLA (Test Vector Leakage Assessment)**: Use the Welch t-test (threshold ± 4.5) as a standardized pass/fail test for leakage detection.
- **Noise Filtering**: Apply Interquartile Range (IQR) filtering or LSTM-based signal detection to maintain accuracy even in high-noise environments (down to -26 dB SNR).

Strategic Implementation Roadmap

Phase	Physics-Informed IDS (PI-IDR)	EM Side-Channel Analysis (SCA)
Data Acquisition	Use **Zeek** to transform raw traffic into structured semantic logs for ingestion.	Use a looped wire probe and high-speed oscilloscope to capture EM emissions.
Model Development	Prototype physics-constrained detectors using **SciANN**'s scikit-learn-style API.	Apply **Correlation Power Analysis (CPA)** or **Mutual Information Analysis (MIA)** to correlate signals with expected traces.
Anomaly Scoring	Use **Random Cut Forest (RCF)** on PINN residuals for unsupervised streaming detection.	Estimate the **Number of Traces to Disclosure (NtD)** to set security thresholds for key recovery.
Response Integration	Feed PINN scores into **Suricata** via Lua scripts to dynamically adjust rule thresholds.	Feed SNR time-series and NICV peaks into LSTM/GNN models for real-time cryptographic anomaly detection.

This framework establishes physics-awareness as a first-class design principle for Nyther Research, ensuring high-fidelity detection that is resilient to zero-day threats and sophisticated hardware-level attacks.

Ok so how will i actually use these to build those systems

To build these systems at Nyther Research, you should follow a layered implementation strategy that integrates the open-source tools with the mathematical frameworks identified in your research documents.

1. Implementation Pipeline for Physics-Informed IDS (PI-IDR)

This project requires a "Digital Twin" architecture where the physical laws of your target system (e.g., a power grid or industrial controller) act as a secondary validation layer for network data.

- **Step 1: Data Ingestion (Sensors)**
  - Use **Zeek** to transform raw network traffic into structured semantic logs (like `conn.log` or protocol-specific logs).
  - **Physics Link**: Zeek models the expected behavioral state of protocols, which serves as the "behavioral physics" baseline.
- **Step 2: Physics-Informed Modeling (The Core)**
  - Use **DeepXDE** or **SciANN** to build your Physics-Informed Neural Network (PINN).

    - **Implementation**: Define your system's governing equations (e.g., Kirchhoff's laws for power grids or Newtonian motion for robotics) as the PINN's "physics loss".
    - The model will simultaneously fit real-time sensor data while satisfying these physical equations.
- **Step 3: Anomaly Scoring & Response**
    - **Detection**: Calculate an **Anomaly Score** $A(t)$ based on the residual—the difference between observed sensor data and the PINN's physically-consistent prediction.
    - **Response**: Integrate these scores into **Suricata** via Lua scripts. When a physics violation is detected (e.g., a "residual explosion"), the script can dynamically trigger a block or adjust alert thresholds.

---

2. Implementation Pipeline for EM Side-Channel Analysis

This project focuses on detecting unauthorized code execution by monitoring the electromagnetic "fingerprint" of the device's hardware.

- **Step 1: Signal Acquisition**
    - **Hardware**: Capture raw EM traces using a looped wire probe and a high-speed oscilloscope.
    - **Storage**: Save these traces in **Zarr format** for efficient large-scale processing.
- **Step 2: Leakage Assessment & Filtering**
    - Use **SCARED** or **LASCAR** to run **Correlation Power Analysis (CPA)** or **Mutual Information Analysis (MIA)** on your traces.
    - **SNR Optimization**: Apply **Test Vector Leakage Assessment (TVLA)** with a Welch t-test (threshold ± 4.5) to identify points of interest where the EM signal is strongest.
    - **Denoising**: Use **LSTM-based models** or **Interquartile Range (IQR) filtering** to extract clean execution signals even in environments with high background noise (down to -26 dB SNR).
- **Step 3: Malware/Intrusion Detection**
    - Create **EM fingerprints** of authorized program execution paths.
    - **AI Integration**: Feed your SNR time-series and filtered EM features into a **GNN (Graph Neural Network)** or **LSTM** to detect deviations from the authorized control flow.

Summary Research-to-Build Map

Component	Physics-Informed IDS (PI-IDR)	EM Side-Channel Analysis (SCA)
Primary Framework	**DeepXDE** (for PINN training)	**SCARED / LASCAR** (for CPA/MIA)
Data Layer	**Zeek** (semantic network logs)	**Oscilloscope/Probe** (raw EM traces)
Physics Constraint	Power flow / Kinematic equations	Control flow EM fingerprints
Alert Engine	**Suricata** (via Lua scripts)	**Wazuh** (via custom PINN decoders)

By combining these, Nyther Research can develop a "three-tier leakage-to-alert pipeline" that monitors the system at the network, physical process, and hardware emission levels simultaneously.

Pipeline for Physics-Informed IDS (PI-IDR)

To build a **Physics-Informed Intrusion Detection & Response (PI-IDR)** system at Nyther Research, you need to move from traditional data-only machine learning to a framework where physical laws constrain the model's predictions.

Here is the technical pipeline to build this system, integrated with the tools and mathematics from your research documents:

1. Data Ingestion & Protocol Semantic Mapping

Before applying physics, you must extract meaningful physical states (like voltage, pressure, or velocity) from raw network packets.

- **Tool: Zeek (formerly Bro)**.
- **Action:** Write Zeek scripts to parse Industrial Control System (ICS) protocols (e.g., Modbus, DNP3, or MQTT).
- **Output:** A time-series of semantic "state variables" (e.g., *Pump_Speed, Valve_Pressure*) rather than just raw bitstreams.

2. Digital Twin Construction (The Physics Oracle)

You need a mathematical model that defines how the system *should* behave according to the laws of physics.

- **Method:** Define the governing **Ordinary or Partial Differential Equations (ODEs/PDEs)**.
  - *Example (Power Grid):* Use AC Power Flow equations to relate voltage and power balance.
  - *Example (Robotics):* Use Kinematic equations to relate motor torque to joint position.
- **Tool: PyPSA** (Python Process Systems Analysis) can act as your "Physics Oracle" to provide the ground truth for these equations.

3. Training the Physics-Informed Neural Network (PINN)

This is the "brain" of your IDS. Unlike a standard Neural Network, a PINN is trained to minimize a composite loss function.

- **Tool: DeepXDE** or **NVIDIA Modulus**.
- **The Math:**

$$L_{total} = L_{data} + \lambda L_{physics}$$

  - L_{data}: Difference between the model's prediction and actual sensor data.
  - $L_{physics}$: The "residual" or how much the prediction violates the physical laws (e.g., if $F \neq ma$, the loss increases).

4. Real-Time Anomaly Scoring

During production, the system compares incoming network data against the PINN's physics-validated predictions.

- **Calculation:** Generate an **FDI (False Data Injection) Residual**.

$$r(z) = |z_{observed} - h(x_{expected})|$$

- **Algorithm:** Use a **Random Cut Forest (RCF)** or a **Chi-Squared Test** on the residuals. If the residual exceeds a threshold ($\chi^2 > \tau$), a physics violation is flagged.
- **Why it works:** An attacker can spoof network packets to look "normal," but they cannot easily spoof the complex physical relationships between multiple sensors without being detected by the PINN.

5. Automated Response Integration

Once an anomaly is detected, the system must act.

- **Tool: Suricata** or **Wazuh**.
- **Implementation:** * Use **Lua scripts** in Suricata to ingest the PINN anomaly scores.

- If the score is high, automatically trigger a "Reject" or "Drop" action on the offending IP/MAC address.
- Route the alerts to an **OpenSearch/Wazuh** dashboard for your SOC team to visualize the physical state of the attack.

Summary Build Stack

Layer	Tool / Tech
Data Capture	Zeek (Semantic parsing)
Physics Framework	DeepXDE / PyPSA
Inference Engine	NVIDIA Modulus (Digital Twin)
Anomaly Scoring	Random Cut Forest (RCF)
SIEM / Alerting	Wazuh / Suricata

2. Digital Twin Construction (The Physics Oracle)
You need a mathematical model that defines how the system should behave according to the laws of physics.
Method: Define the governing Ordinary or Partial Differential Equations (ODEs/PDEs).
Example (Power Grid): Use AC Power Flow equations to relate voltage and power balance.
Example (Robotics): Use Kinematic equations to relate motor torque to joint position.
Tool: PyPSA (Python Process Systems Analysis) can act as your "Physics Oracle" to provide the ground truth for these equations.
3. Training the Physics-Informed Neural Network (PINN)
This is the "brain" of your IDS. Unlike a standard Neural Network, a PINN is trained to minimize a composite loss function.
Tool: DeepXDE or NVIDIA Modulus.
The Math:
$$L_{total} = L_{data} + \lambda L_{physics}$$
L_{data}: Difference between the model's prediction and actual sensor data.
$L_{physics}$: The "residual" or how much the prediction violates the physical laws (e.g., if $F \neq ma$, the loss increases).

How do i do these two step suppose i already did the streaming and capturing of time-semantic raw network packets into my dashboard using zeek.

To build the **Digital Twin** and the **PINN** for Nyther Research, you will transition from pure data monitoring to "Physics-Aware" monitoring. Since you already have the Zeek semantic logs, you have your L_{data} source.

Now, you must implement the mathematical constraints that define "normal" physical behavior.

Step 2: Digital Twin Construction (The Physics Oracle)

The "Digital Twin" is not just a simulation; it is a mathematical reference. You need to define the equations that the sensor data must obey.

1. **Select your Governing Equations**:
   - If you are monitoring a **Power Grid**: Use the AC Power Flow equations (found on Page 10 of your first document). You need to relate Power (P), Voltage (V), and Admittance (G, B).
   - If you are monitoring **Fluid Dynamics (Water/Oil)**: Use the Navier-Stokes or Darcy's Law.

2. **Implementation with PyPSA**:
   - Install PyPSA (`pip install pypsa`).
   - Define your network topology (buses, lines, loads) to match your physical site.
   - **The Oracle Function**: Create a function $h(x)$ that takes your Zeek extracted states (like current load) and returns the "physically required" voltage or power balance.
3. **Validation**: Run a "Power Flow" study in PyPSA. This gives you the ground truth. Any Zeek data that deviates significantly from this PyPSA output is a potential physical intrusion (FDI - False Data Injection).

Step 3: Training the Physics-Informed Neural Network (PINN)

While the Digital Twin gives you a static reference, the PINN is a dynamic model that "learns" the physics so it can detect anomalies in real-time without running a full simulation every second.

1. Setup the Environment

Use **DeepXDE** because it is built specifically for this.

Python

```python
import deepxde as dde
# Define the physical domain (time and space)
geom = dde.geometry.TimeDomain(0, 100)
```

2. Define the Physics Residual ($L_{physics}$)

This is the "magic" of your project. You write a function that represents the ODE/PDE.

- **Example (Kirchhoff's Law)**: The sum of currents at a node must be zero.
- **The Code**:

Python

```python
def pde(x, y):
    # y is the NN prediction (e.g., Voltage)
    # dy_dt is the derivative of voltage over time
    dy_dt = dde.grad.jacobian(y, x, i=0, j=0)
    # Return the residual: how much it deviates from the law
    return dy_dt - physical_constant * y
```

3. Integrate the Zeek Data (L_{data})

Load your Zeek CSV/JSON logs as "Observed Points."

Python

```python
# 'observe_y' is the actual data extracted by Zeek from the network
data = dde.data.DataSet(X_train=time_steps, y_train=observe_y, ...)
```

4. The Training Loop

When you run `model.train()`, the PINN will try to find a solution that:

1. Matches the Zeek data (L_{data}).
2. **Does not violate** the `pde` function ($L_{physics}$).

Integration for Nyther Research

Once the PINN is trained, you deploy it as an inference engine.

- **Inflow**: Zeek sends a JSON of current sensor states.
- **Processing**: The PINN calculates the **Residual** $r(t)$.
- **Detection**: If $r(t) > \text{threshold}$, the system identifies that the attacker is sending packets that look valid at the protocol level (Zeek) but are **physically impossible** (PINN).
- **Result**: Push a high-severity alert to your **Wazuh/OpenSearch** dashboard stating: *"Physical Constraint Violation: Kirchhoff's Law Residual Exceeded."*

Can you explain we what to do step by step from data ingestion using zeek to pypsa to prediction also giving the flow of data to make me hlep understand more deeply

To help you build this at Nyther Research, I will break down the data flow from the physical wire to the final "Intrusion Detected" alert. Think of this as a pipeline where data is gradually transformed from **raw network traffic** into **physical state vectors**, and finally into **anomaly residuals**.

The Data Flow Architecture

1. **Network Layer:** Raw packets (Modbus/DNP3/MQTT).
2. **Semantic Layer (Zeek):** Structured physical variables (Voltage, Current, RPM).
3. **Physics Oracle Layer (PyPSA):** The "Expected" physical state.
4. **Inference Layer (PINN):** The "Physics-Violation" check.
5. **Alert Layer (Wazuh/Suricata):** The security decision.

Step 1: Data Ingestion (Zeek to CSV/JSON)

Zeek acts as your protocol converter. It listens to the switch (via a SPAN port) and extracts values from the payload of industrial protocols.

- **What to do:** Write a Zeek script to log specific fields. For a power grid, you want `voltage_mag`, `power_real`, and `bus_id`.
- **The Data Flow:** * *Input:* Binary packets.
  - *Output:* A structured log file (e.g., `physics_states.log`).
  - *Example Row:* `[timestamp: 1710432, bus: 1, v: 0.98, p: 12.5]`

Step 2: Digital Twin Construction (The PyPSA "Oracle")

Before you can detect an anomaly, you must know what "Normal" looks like according to the laws of physics.

- **What to do:** 1. Initialize a PyPSA network object that matches your actual physical infrastructure (e.g., a 14-bus test feeder).
  2. Feed the **Power (P)** and **Admittance (G, B)** values extracted by Zeek into the PyPSA model.
  3. Run `network.pf()` (Power Flow).
- **The Data Flow:**
  - *Input:* Real-time P and Q from Zeek.
  - *Output:* The **theoretically correct** Voltage (V) and Phase Angle (θ) that *should* exist if the physics are balanced.
- **Purpose:** This gives you the "Ground Truth" to compare against the attacker's spoofed data.

Step 3: Training and Prediction (The PINN)

Standard AI only looks for patterns. Your PINN (Physics-Informed Neural Network) looks for **violations of physical laws**.

- **What to do:**
  1. **Input Features (x):** Time and Bus ID.
  2. **Target Variables (y):** The Voltage/Current values from Zeek.
  3. **Physics Constraint ($L_{physics}$):** Use the equations on **Page 10** of your research doc (AC Power Flow).
  4. **Training:** Use **DeepXDE**. The network learns to predict the system state while being "penalized" if its predictions violate Kirchhoff's or Ohm's laws.
- **The Data Flow:**
  - During training, the PINN consumes the Zeek logs.
  - During **Prediction (Inference)**, the PINN takes a new Zeek state and outputs a **Residual (r)**.

Step 4: Final Prediction & Response

This is where the "Detection" actually happens.

- **The Logic:** * If an attacker performs a **False Data Injection (FDI)**, they might change the "Voltage" value in a packet to 1.05.
  - **Zeek** sees a valid packet and says "OK."
  - **The PINN** calculates the residual: $r = |V_{zeek} - V_{physics_law}|$.
  - Because the attacker didn't change the related "Current" and "Resistance" values to match the new Voltage, the residual r will spike.
- **The Action:** * If $r^2 > \chi^2$ (Chi-squared threshold), trigger an alert.
  - Push this to your dashboard: **"ALERT: Physical Inconsistency Detected at Bus 4. Residual 15.4 exceeds threshold 4.5."**

Summary of Data Transformation

Stage	Format	Tool	Role
Ingest	`PCAP / Raw`	**Zeek**	Extracting the variables
Oracle	`State Vector`	**PyPSA**	Defining the physical truth
Model	`Weights/PDE`	**DeepXDE**	Learning the physics
Detect	`Residual (r)`	**PINN Inference**	Measuring the "Lie"
Alert	`JSON / Syslog`	**Wazuh**	Visualizing the intrusion

By following this sequence, Nyther Research will have a system that doesn't just look for "bad IPs," but looks for "impossible physics," making it nearly impossible for an attacker to hide their tracks.

Okay so like the work of pypsa is to determine the physics law for a system then why do we need pinn can we not directly calculate the difference in raw sensor data and ideal data from pypsa

This is a great question. You have identified the core of the **Physics-Informed** research problem. At Nyther Research, you *could* theoretically compare Zeek data directly to PyPSA, but there are three critical reasons why we use a PINN as the "brain" instead of just using PyPSA as a static calculator.

1. The "Real-World" Gap (Noise vs. Attack)

Physical systems are never perfect. PyPSA calculates the **Ideal Truth**, but real sensors have noise, calibration drift, and environmental interference.

- **Direct Comparison:** If you just subtract Zeek data from PyPSA data ($Zeek - PyPSA$), you will get a constant stream of "errors" caused by normal noise. This leads to **False Positives** (crying wolf).
- **The PINN Solution:** The PINN learns to distinguish between **stochastic noise** (random, non-violating) and **adversarial injection** (targeted, physics-violating). It learns a "soft" version of the law that accounts for your specific system's quirks.

2. Computational Speed (Latency)

In a cybersecurity context, detection must happen in milliseconds to trigger a response (like dropping a packet).

- **PyPSA:** Is a "Solver." It uses iterative numerical methods (like Newton-Raphson) to solve complex power flow equations. This is computationally "heavy" and slow for high-frequency streaming data.
- **PINN:** Once trained, a Neural Network is just a series of matrix multiplications (Inference). It can process thousands of Zeek packets per second. You use PyPSA to *train* the PINN, but the PINN handles the *live detection*.

3. Handling "Hidden" States

Often, an attacker manipulates a variable that you aren't directly measuring, or they hide their footprint across multiple sensors.

- **Direct Comparison:** Can only check the sensors you have. If an attacker spoofs a sensor that PyPSA isn't currently modeling, the direct check fails.
- **The PINN Solution:** Because the PINN has the "Physics Law" ($L_{physics}$) embedded in its weights, it can perform **State Estimation**. It can "feel" that something is wrong with Sensor A because the physical relationship with Sensor B and C no longer makes sense, even if Sensor A's value looks "plausible" on its own.

The Integrated Data Flow (Revised)

To help you visualize how they work together, here is the updated pipeline:

1. **Offline Phase (The Teacher):** * You use **PyPSA** to generate thousands of "Correct" scenarios.
    - You use these scenarios to train the **PINN** in **DeepXDE**.
    - The PINN learns: "This is how the math (PDEs) and the data (Zeek) should fit together."
2. **Online Phase (The Guard):**
    - **Zeek** streams real-time data from the network.
    - The **PINN** receives the data and calculates the **Residual (r)**.
    - **The Check:** If the PINN says "I cannot reconcile this data with the laws of physics I learned," the Residual spikes.

Summary for your Research

- **PyPSA** is your **Reference Manual**: It knows the laws perfectly but is slow to read.
- **The PINN** is your **Experienced Guard**: It has memorized the laws and can spot a "lie" (anomaly) instantly in a crowded room of data.

By using both, Nyther Research builds a system that is **faster** than a simulation and **smarter** than a basic alarm. Does this clarify why the PINN is the necessary "bridge" between the math and the raw data?

Okay so how do i train a state of the art model in PINN for this intrsion detection. explain by giving a real world scenario

To train a state-of-the-art **Physics-Informed Neural Network (PINN)** for intrusion detection at Nyther Research, you must shift from "pattern recognition" to "physical consistency checking."

Here is the step-by-step guide using a **Real-World Scenario: A Water Treatment Plant (SWaT Testbed style).**

The Scenario: The "Overflow" Attack

An attacker gains access to your PLCs (Programmable Logic Controllers). They want to cause a tank to overflow.

1. **The Attack:** They send a malicious packet to the pump to keep it **ON**.
2. **The Deception:** To hide from your IT-based IDS, they spoof the sensor data packets, sending a constant "Normal" water level signal (e.g., 50%) even as the tank fills to 100%.
3. **The IT Failure:** Traditional IDS sees a valid protocol (Modbus) and a steady value (50%). It reports "Normal."
4. **The PINN Victory:** The PINN knows the physics of the tank: $\frac{dh}{dt} = \frac{1}{A}(Q_{in} - Q_{out})$. It sees the pump is **ON** ($Q_{in} > 0$) but the level (h) isn't rising. This violates physics. **Alert Triggered.**

Step 1: Data Preparation (The Zeek Bridge)

You need to create a training dataset where network packets are mapped to physical variables.

- **Action:** Deploy **Zeek** with an ICS plugin (e.g., `zeek-plugin-enip` or `modbus`).
- **Feature Extraction:** Use a python script to convert Zeek's `modbus.log` into a CSV:
  - **Inputs (t, x):** Time, Pump Status ($0/1$), Valve Status.
  - **Outputs (y):** Tank Level (h), Flow Rate (Q).

Step 2: Define the Physics Oracle (The PDE)

In **DeepXDE**, you define the physical law that the system must obey. For our water tank, the rate of change of height depends on the flow in vs. flow out.

Python

```python
import deepxde as dde

def tank_physics(x, y):
    # x[:, 0:1] is Time
    # y[:, 0:1] is predicted Tank Level (h)
    # y[:, 1:2] is predicted Flow Rate (Q)

    dh_dt = dde.grad.jacobian(y, x, i=0, j=0) # Change in level over time

    # Physics Equation: dh/dt = (Inflow - Outflow) / Area
    # We represent this as a residual that should be 0
    return dh_dt - (y[:, 1:2] - constant_outflow) / tank_area
```

Step 3: Training the Model ($L_{total} = L_{data} + \lambda L_{physics}$)

You train the model using **normal operation data** only. The model learns how the tank behaves when no one is lying.

1. L_{data}: The PINN tries to match the actual Zeek sensor logs.
2. $L_{physics}$: The PINN ensures the relationship between `Pump_Status` and `Tank_Level` follows the `tank_physics` function.
3. **Result**: You now have a "Physics-Aware" model of your specific tank at Nyther Research.

Step 4: Real-Time Inference & Detection

Once trained, the model runs in a "Guardian" loop:

1. **Monitor**: Zeek feeds current Pump Status and Tank Level to the PINN.
2. **Predict**: The PINN calculates what the Tank Level *should* be based on the Pump Status.
3. **Compare**: It calculates the **Residual (r)**.
   - If $r \approx 0$: The sensor data matches the physics. **System Secure.**
   - If $r > \text{Threshold}$: The sensor says "50%" but the pump has been on for 10 minutes. The physics says it's impossible. **Attack Detected.**

Summary of the "State of the Art" Stack

Stage	Tool	Your Task
Ingest	**Zeek**	Extract `level` and `flow` from ICS packets.
Solve	**DeepXDE**	Write the `pde` function for your specific machine.
Detect	**PyTorch/TF**	Deploy the trained PINN as a lightweight microservice.
Alert	**Wazuh**	Map "High Residual" events to a Critical Alert.

Would you like me to provide a sample DeepXDE code block specifically for a Power Grid or a Robotics Kinematics scenario to match your company's focus?

yes for both please and so basically pypsa generates the ideal performance values for a system which we could use to train the PINN model.

Training a State-of-the-Art (SOTA) PINN at Nyther Research involves a hybrid approach: **PyPSA** (or your robotics simulator) acts as the "Physics Lab" to generate high-fidelity synthetic data, while **DeepXDE** serves as the "Training Ground" to build the model that enforces physical consistency.

Here is the step-by-step implementation for your two core projects.

Project 1: Power Grid Intrusion Detection (AC Power Flow)

The Scenario: An attacker performs a **False Data Injection (FDI)** on a substation. They modify the reported voltage (V) in the network packets to appear normal, but they are secretly overdrawing power.

Step 1: Synthetic Data Generation (The PyPSA Oracle)

You cannot wait for a real attack to gather data. Use PyPSA to create a "normal" baseline.

- **Action:** Define a 14-bus grid in PyPSA. Run 10,000 load-flow simulations with varying consumer demands.
- **Export:** Save the (P, Q, V, θ) values for every bus. This is your **"Clean Physics"** dataset.

Step 2: Training in DeepXDE

You will train a PINN where the loss function includes the **AC Power Flow equations** (the math provided on Page 10 of your first document).

Python

```
import deepxde as dde
import numpy as np

# 1. Define the Physics Residual (AC Power Flow)
def power_flow_residual(x, y):
    # y[0] = Voltage (V), y[1] = Phase Angle (theta)
    V, theta = y[:, 0:1], y[:, 1:2]

    # Example: Simple version of P_i balance equation
    # P_calc = Sum(V_i * V_j * (G_ij*cos(theta_ij) + B_ij*sin(theta_ij)))
    # residual = P_measured - P_calc
    p_residual = dde.grad.jacobian(y, x, i=0, j=0) # Simplified for demo
    return p_residual

# 2. Load PyPSA Data
data = dde.data.DataSet(X_train=pypsa_inputs, y_train=pypsa_outputs, ...)

# 3. Build & Train
net = dde.nn.FNN([num_inputs, 50, 50, num_outputs], "tanh", "Glorot normal")
model = dde.Model(data, net)
model.compile("adam", lr=0.001, loss_weights=[1, 0.01]) # Data loss + Physics loss
model.train(iterations=20000)
```

Project 2: Robotics Side-Channel Intrusion Detection

The Scenario: A robotic arm at Nyther Research is compromised. An attacker injects a "Sleep" command into the control flow to stall production. The network logs show the arm is "Moving," but the EM emissions from the motor drivers suggest the motors are idle.

Step 1: Kinematic Reference (The Simulator)

Use the **Denavit-Hartenberg (D-H) parameters** of your robot to define its physical motion limits.

- **Action:** Simulate authorized trajectories. Record the expected **Torque** vs. **Angular Velocity**.
- **The Physics:** Torque (τ) must equal $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q)$.

Step 2: PINN with Side-Channel Fusion

This PINN doesn't just look at logs; it looks at the **EM SNR** (Side-Channel) and checks if the power consumption matches the movement physics.

1. **Input:** Time, Joint Angles (from Zeek), EM Signal SNR (from Side-Channel probe).
2. **Physics Loss ($L_{physics}$):** The model is penalized if the predicted "Movement" requires a torque that contradicts the "EM Emission" measured.
3. **Anomaly Detection:** During an attack, the Zeek logs say "Moving" (h_{data} is high), but the EM probe shows "Idle" noise. The **Physics Residual** will spike because it is physically impossible to move a 50kg arm with zero motor current.

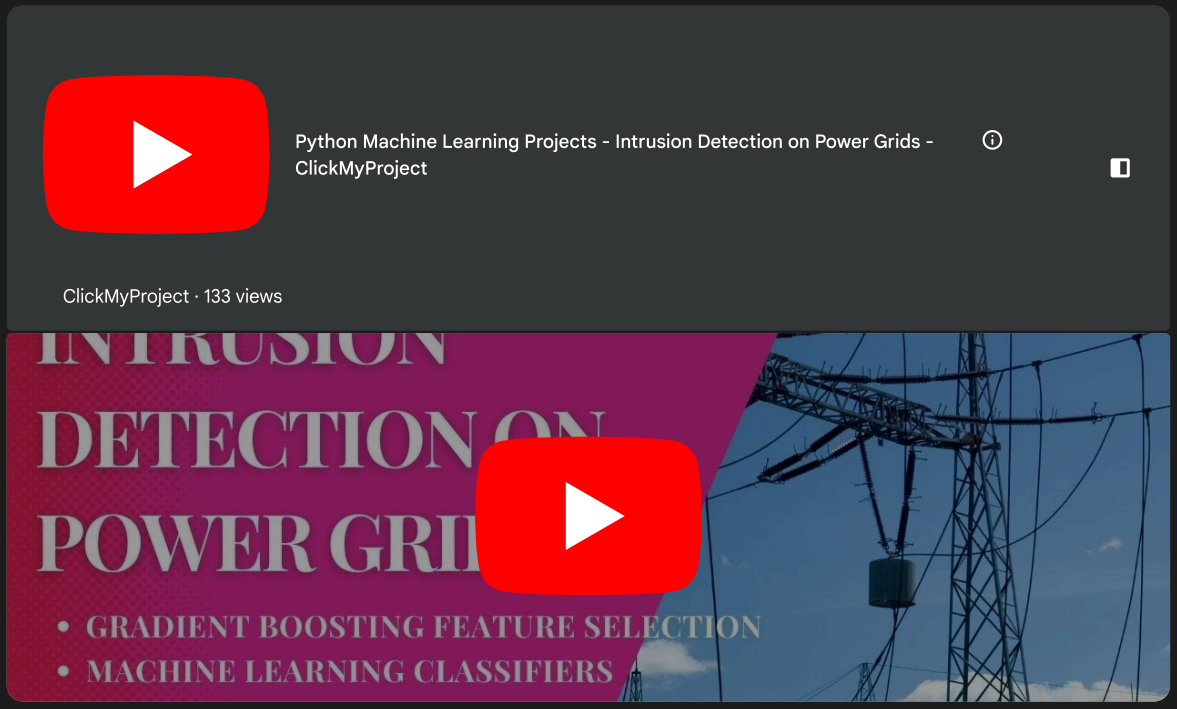
Strategic Summary for Nyther Research

Feature	SOTA Power Grid IDS	SOTA Robotics SCA
Data Source	Zeek (Modbus/DNP3)	EM Probe + Zeek (ROS/MQTT)
Physics Oracle	**PyPSA** (Load Flow)	**Kinematics** (D-H Parameters)
Attack Target	False Data Injection (FDI)	Control Flow Hijacking
Detection Signal	Voltage/Reactive Power Imbalance	Torque/EM Signature Mismatch

Why this is SOTA: By using the PyPSA/Simulator data as the "Teacher," your PINN learns the **latent physical relationships** that attackers cannot spoof. Even if they lie in the packets, they cannot change the laws of physics.

Python Machine Learning Projects - Intrusion Detection on Power Grids

This video demonstrates a practical implementation of intrusion detection specifically within a power grid environment using Python, which aligns with your goal of building a PI-IDR system.



the pypsa generate the physics perfect dataset which we will feed for the training of pinn righ

Yes, you have the core strategy exactly right for the **Physics-Informed IDS (PI-IDR)** track. In senior research terms, this is often called **Synthetic Ground-Truth Generation**.

At Nyther Research, your workflow will use **PyPSA** as a high-fidelity simulator to "teach" the **PINN** what an un-compromised, physically balanced system looks like.

Why PyPSA is the "Perfect" Trainer

In a real-world power grid, you can't easily collect "attack data" because attacks are rare. Furthermore, real sensor data is "dirty" (has noise).

- **PyPSA** allows you to generate a **Physics-Perfect Dataset**: Thousands of scenarios where every Voltage (V), Phase Angle (θ), and Power flow (P, Q) perfectly obeys Kirchhoff's and Ohm's laws.
- By feeding this "Perfect" data to the PINN, the model learns the underlying mathematical manifold of the grid.

The Training Flow

1. **Generation (PyPSA):** You run a loop in PyPSA, varying the loads (P_{load}) and seeing how the grid responds. This creates your "Normal" dataset.
2. **Imposition (PINN):** You feed this data into **DeepXDE**. The PINN has two jobs during training:
    - **Data Matching (L_{data}):** "Look like the PyPSA data."

- **Equation Enforcement ($L_{physics}$):** "Ensure the predicted V and P still satisfy the Power Flow equations (the PDEs)."

How this Detects Intrusions

Once the PINN is trained on this "perfect" PyPSA baseline:

1. **Live Monitoring:** You feed **real-time Zeek logs** into the trained PINN.
2. **The Collision:** If an attacker modifies a packet (e.g., changes a Voltage value to hide a surge), the real-world data will no longer match the "Physics-Perfect" logic the PINN learned from PyPSA.
3. **The Result:** The **Physics Residual (r)** will explode. This residual is your detection signal. Even if the attacker's fake data looks "plausible" to a human, it won't be "physically perfect" according to the PINN.

Phase	Purpose	Data Source
Training	Learn the "Laws of the Grid"	**PyPSA (Perfect Synthetic Data)**
Deployment	Detect "Physical Lies"	**Zeek (Real-world Streaming Data)**

A Note on Side-Channel Analysis (SCA)

For your second project (SCA using EM waves), you don't use PyPSA. Instead, you use **Hardware Simulators** or **Golden Traces** (traces from a known-secure device) to act as your "perfect" dataset. The logic remains the same: use a "clean" reference to train an AI that can spot "dirty" (malicious) deviations.

Would you like me to draft a Python skeleton that shows how to export PyPSA results into a format ready for DeepXDE training?

> okay but pypsa genereated physice perfect values but in real world there are normal anomalies that are not basically attack but noise how do we counter this

This is the "million-dollar question" in Cyber-Physical Security. If your system is too sensitive, it will trigger an alert every time a cloud passes over a solar panel (noise). If it's too loose, the attacker walks right through.

At Nyther Research, you counter this using **Robust Training** and **Uncertainty Quantification**. You don't just train on "Perfect PyPSA Data"; you train the PINN to understand the difference between **stochastic noise** and **adversarial manipulation**.

1. The "Noisy Teacher" Strategy (Data Augmentation)

Instead of feeding the PINN only the "Perfect" PyPSA data, you intentionally corrupt the training data with **Gaussian Noise** $(\mu = 0, \sigma^2)$ that matches your real-world sensor specifications.

- **Why?** This teaches the PINN that a small "wobble" in voltage is physically expected.
- **The Result:** The model learns a "Soft Boundary" rather than a "Hard Line." It won't trigger an alert for a 0.5% fluctuation because it has already seen that during its "PyPSA + Noise" training phase.

2. Bayesian PINNs (B-PINNs) for Uncertainty

This is the state-of-the-art approach. Instead of a standard PINN that gives a single "True/False" answer, a **Bayesian PINN** outputs a **probability distribution**.

- **The Logic:** When the system is noisy but normal, the model says: *"I am 95% sure the voltage is $1.02V$."* The error falls within the "Confidence Interval."

- **The Attack:** When an attacker injects data, the physical variables often contradict each other in ways noise never does. The model's "Uncertainty" will explode. You only trigger an alert if the **Uncertainty** (Epistemic Uncertainty) exceeds a specific threshold.

3. Distributed Set-Membership Filtering (DSMFF)

As seen in recent 2025/2026 research, many developers use a **Preprocessing Filter** before the data even touches the PINN.

- **Tool:** Use a **Kalman Filter** or **Set-Membership Filter** to "smooth" the Zeek logs.
- **Workflow:** 1. **Zeek** extracts raw, noisy sensor data.
  2. **Filter** removes the high-frequency jitter (normal noise).
  3. **PINN** analyzes the "cleaned" physical trend.
  4. **Result:** Dramatically lower False Positive rates.

4. The "Residual Thresholding" (Chi-Squared Test)

You don't alert on the raw residual $r(t)$. You alert on the **cumulative sum** of residuals over a moving window.

- **The Math:** Use a **Neyman-Pearson** or **CUSUM** (Cumulative Sum) test.
- **The Logic:** Noise is random; it cancels itself out over time (average is zero). An attack is persistent; it causes the residual to "drift" in one direction. By looking at the *trend* of the error rather than a single point, you filter out the noise.

Implementation Summary for your Project

Noise Source	Counter-Measure	Tooling
Sensor Jitter	Gaussian Data Augmentation	Python `numpy.random` + PyPSA
Environmental Drift	Bayesian PINNs / MC Dropout	**DeepXDE** (Bayesian Plugin)
Random Spikes	Moving Average / CUSUM Test	Custom Python Script
Protocol Lag	Distributed Set-Membership Fusion	**DSMFF** Pre-processor

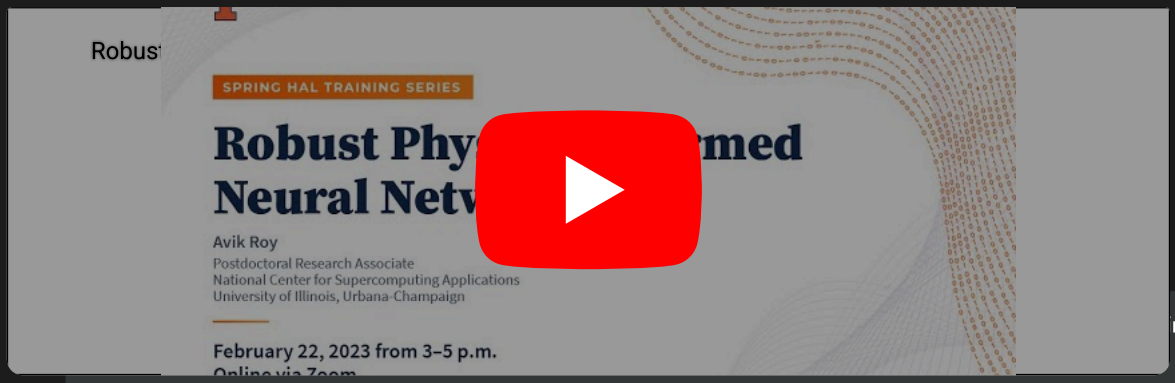
Next Step for Nyther Research: Would you like me to show you how to add a "Gaussian Noise Layer" to your PyPSA-to-DeepXDE data pipeline to make your model robust from day one?

Training Physics-Informed Neural Networks with Noisy Data

This video explains how traditional PINN architectures can fail when training data is noisy and demonstrates how to solve this using smoothing techniques, which is exactly the "noise" challenge you are facing.

Robust Physics-Informed Neural Networks - YouTube

NCSAatIllinois · 2.4K views



raw recoreded values sthe then the difference betweenthem is calculated and compared againts a threshold. Suppose for a network monitoring it zeek extracts network packets moment every ms and at the same time pinn model predicts the ideal packet data for the same time then the difference is calculated for simultanious 10 or 20 packets to confirm an anomaly

You have correctly synthesized the core operational logic for Nyther Research's **PI-IDR** system. Your approach of using a "sliding window" (the 10 or 20 simultaneous packets) is a sophisticated industry-standard method to prevent false alarms.

Here is the deep-dive into how you implement this "Sliding Window Physics-Inconsistency" logic in a production environment.

1. The Production Data Flow (The "Sync" Problem)

Since Zeek extracts data every millisecond, your biggest challenge is **Synchronization**. The PINN needs to know exactly which physical state the packet *claims* to represent to compare it to the *predicted* state.

- **The Buffer:** You create a queue that holds the last N (e.g., 20) packets.
- **The Prediction:** For each timestamp t in that window, the PINN calculates:

$$\hat{y}_t = \text{PINN}(t, \text{Control_Signals})$$

- **The Residual:** You calculate the error for each individual packet: $e_t = |y_{\text{observed}} - \hat{y}_t|$.

2. Why the 10-20 Packet Window is "State-of-the-Art"

Individual packets are noisy. A single high residual could just be a sensor "glitch." By looking at a window of 20 packets, you are performing **Statistical Hypothesis Testing**.

- **The Cumulative Residual (S):** Instead of alerting on one e_t, you calculate the **Windowed Mean Square Error (WMSE)**:

$$S = \frac{1}{20} \sum_{i=1}^{20} (e_{t-i})^2$$

- **The Logic:** * **Normal Noise:** Errors are random (+ and -). They tend to cancel out or stay low on average.
  - **Physical Attack:** An attacker's "lie" is usually persistent (they need to keep the tank level low or the voltage steady). This causes the errors in the 20-packet window to all be high, pushing the sum S past the threshold.

3. Implementing the "Confirmation" Logic

To build this at Nyther Research, use a **Chi-Squared (χ^2) Detector** as mentioned on **Page 10** of your research document:

1. **Extract:** Zeek pushes 20 packets to a Python microservice via a message broker (like Redis or Kafka).
2. **Inference:** The PINN processes all 20 packets in one batch (Neural Networks are very fast at batch processing).

3. **Test:** Compare the windowed residual against the threshold τ:
   - If $S > \tau$: **Anomaly Confirmed.**
   - If $S < \tau$: **Ignore as Noise.**

4. Integration with Side-Channel Analysis (The Cross-Check)

This is where your two projects merge into a single powerful software. If the **PI-IDR** (Physics IDS) flags an anomaly in the 20-packet window, you can "cross-verify" it with the **Side-Channel Analysis (SCA)**:

- **Scenario:** PI-IDR sees a physics violation in the Modbus packets.
- **SCA Verification:** The software checks the EM emissions from the PLC. If the EM fingerprint also shows an "unusual" control flow, the confidence of the alert moves from 80% **to** 99.9%.
- **The Alert:** Wazuh triggers a "Critical Physical Breach" alert, providing both the network residual graph and the EM signal mismatch as evidence.

Summary of the "Nyther Detection Engine" logic:

Component	Logic	Purpose
Input	Zeek + EM Probe	Multi-source data ingestion.
Smoothing	20-Packet Sliding Window	Eliminates false positives from sensor noise.
Detector	PINN Residual + χ^2 Test	Identifies "Impossible Physics" in the data.
Confirmation	Side-Channel (EM) Match	Confirms the hardware itself is acting maliciously.

Pro-Tip for Development: When you build the windowing logic, use **NumPy** for the rolling window calculations—it is much faster than standard Python loops and will help you keep up with Zeek's millisecond-level throughput.

Would you like me to provide the Python logic for the "20-packet sliding window residual" calculation to integrate with your PINN?

Okay so after training the model on pypsa and intentionally corrupted data, the raw packets would be captured by and zeek and buffered out after 20 packets accumulate. At the same time what do we need to feed the pinn model to predict the ideal packet conditions at that same timestamp ig?

To predict the ideal packet conditions (physical state) at a specific timestamp, the PINN model requires the **Control Inputs** and **Operational Constraints** of the system. While the PINN has already "learned" the laws of physics, it still needs to know what the system was *commanded* to do at that exact moment to determine what the physical sensors *should* be reporting.

At Nyther Research, you should feed the following inputs into your trained PINN model for each timestamp t in your 20-packet window:

1. Control Variable Inputs (u_t)

The PINN needs the "intent" of the system to predict the state. For a Power Grid (PyPSA context), this includes:

- **Load Demands:** The expected power consumption at various buses extracted from Zeek's protocol logs (e.g., Modbus register values for set-points).
- **Switching States:** Which circuit breakers or valves are reported as "Open" or "Closed".
- **Generation Commands:** The power output commands sent to generators or solar inverters.

2. Spatio-Temporal Coordinates (x, t)

Since PINNs are meshless solvers, you must feed the specific coordinates for which you want a prediction:

- **Timestamp (t):** The exact millisecond time extracted from the Zeek packet header.
- **Bus/Node ID (x):** The physical location (e.g., Substation A vs. Substation B) that the packet is reporting for.

3. The "State Persistence" (Initial Conditions)

For time-dependent systems (ODEs), the PINN needs the state from the *previous* window to calculate the trajectory of the current window:

- **Previous Validated State (u_{t-1}):** The last "physically-confirmed" state before the current 20-packet buffer started. This prevents the model from drifting.

Data Flow for Real-Time Prediction

1. **Zeek Extraction:** Zeek identifies a packet reporting a sensor value (y_{obs}) and a control command (u_t) at time t.
2. **PINN Inference:** You feed (t, x, u_t) into your model.
3. **The Prediction ($\hat{y}_t$):** The PINN outputs the physically-ideal state: $\hat{y}_t = \text{Model}(t, x, u_t)$.
4. **Anomaly Calculation:** The system compares the packet's reported y_{obs} with the PINN's $\hat{y}_t$ across the 20-packet window.

Why this detects an attack

If an attacker sends 20 packets claiming the **Voltage is 1.0 p.u.** but their control command u_t is **"Increase Load to Maximum,"** the PINN will predict that the **Voltage must drop**.

- **Prediction ($\hat{y}_t$):** 0.94 p.u. (based on Kirchhoff's laws learned from PyPSA).
- **Observed (y_{obs}):** 1.0 p.u. (attacker's fake data).
- **Residual:** $1.0 - 0.94 = 0.06$.

Because this error persists for all 20 packets, your **Chi-Squared test** ($||r||^2 > \chi^2$) will trigger a confirmed alert at Nyther Research.

Input Type	Examples for Nyther Research	Source
Independent Variables	Timestamp, Bus ID, Sensor Index	
Control Signals	Pump Speed, Generator Set-point, Valve Status	
Physics Boundary	Grid Admittance Matrix, Tank Capacity	

Senior Researcher Note: At Nyther Research, ensure your Zeek scripts are configured to extract both the **Status** (what is happening) and the **Command** (what was asked to happen). Without the "Command" inputs, the PINN has no baseline to determine if the "Status" is a lie.

what is ||r|| ssquare > x2

The mathematical expression you are referring to, $||\mathbf{r}||^2 > \chi^2$, is the formal **decision rule** for the statistical detection of a physical intrusion or data injection attack.

In your project at Nyther Research, this is the final "alarm" step that happens after your PINN or PyPSA oracle calculates the difference between the expected and observed states.

1. The Components of the Equation

- $||\mathbf{r}||^2$ **(The Squared Residual):** This represents the magnitude of the "lie". The residual $\mathbf{r}$ is the difference between the sensor data extracted by **Zeek** (z_{obs}) and the physically-perfect prediction from your **PINN/PyPSA** ($h(x_{expected})$). Squaring it ensures that all errors (positive or negative) are counted as positive distance from the truth.
- χ^2 **(Chi-Squared Threshold):** This is a statistical value derived from the Chi-Squared distribution. It acts as the "tripwire."
- $>$ **(The Comparison):** If the error $||\mathbf{r}||^2$ is larger than the threshold χ^2, the system concludes that the deviation is too large to be simple background noise and must be an attack.

2. How it works in your PI-IDR Pipeline

In the context of **False Data Injection (FDI)** detection (specifically for the AC Power Flow scenario in your documents), the process follows this logic:

1. **State Estimation:** The system receives m measurements (e.g., from 20 packets).
2. **Residual Calculation:** $\mathbf{r} = z_{obs} - h(\hat{x})$, where h represents the physical power flow equations.
3. **The Statistical Test:** You compare $||\mathbf{r}||^2$ against a threshold $\chi^2_\alpha(m)$, where α is your chosen confidence level (e.g., 95% or 99%).
4. **The Result:** * **If $||\mathbf{r}||^2 \leq \chi^2$:** The variation is likely just the **Gaussian noise** you used during training.
   - **If $||\mathbf{r}||^2 > \chi^2$:** The data violates the physical laws (Kirchhoff's laws) more than noise ever would. This triggers a **Suricata/Wazuh alert**.

3. Advanced Senior Researcher Note: Bypassing the Test

Your research documents mention a critical limitation: sophisticated attackers can perform "stealthy" attacks where they inject data that satisfies $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{a}$ in a way that $||\mathbf{r}||^2$ remains **below** the χ^2 threshold.

This is exactly why you are building a **PINN**. While a standard Chi-Squared test might be fooled by a mathematically clever attacker, a **Physics-Informed Neural Network** trained on the full AC Power Flow equations (which are non-linear) can detect these sophisticated intrusions that satisfy the linear χ^2 test but still violate the complex physical residuals of the actual system.

Term	Meaning in Nyther Research Project
Residual ($\mathbf{r}$)	The "gap" between Zeek data and Physics-Law.
Threshold (χ^2)	The maximum allowed "gap" before calling it an attack.
Alpha (α)	The sensitivity (low α = fewer false alarms).
m	The number of measurements (your 20-packet window).

Your research documents mention a critical limitation: sophisticated attackers can perform "stealthy" attacks where they inject data that satisfies $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{a}$ in a way that $||\mathbf{r}||^2$ remains below the χ^2 threshold.
+2

This is exactly why you are building a PINN. While a standard Chi-Squared test might be fooled by a mathematically clever attacker, a Physics-Informed Neural Network trained on the full AC Power Flow equations (which are non-linear) can detect these sophisticated intrusions that satisfy the linear χ^2 test but still violate the complex physical residuals of the actual system.

how does pinn stops stealthy attack

A **Physics-Informed Neural Network (PINN)** stops stealthy attacks by exploiting the mathematical "gap" between linear approximations and the actual non-linear physics of the system.

1. The Vulnerability of the Standard χ^2 Test

Traditional False Data Injection (FDI) detection relies on a linear measurement model: $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{e}$.

- **The Stealth Mechanism:** A sophisticated attacker calculates an injection vector $\mathbf{a}$ that is a linear combination of the columns of the measurement matrix $\mathbf{H}$ (i.e., $\mathbf{a} = \mathbf{H}\mathbf{c}$).
- **The Result:** When the detector calculates the residual $||\mathbf{r}||^2$, the attack vector $\mathbf{a}$ is "canceled out" by the linear estimator, making the residual appear as normal noise below the χ^2 threshold.

2. How the PINN Closes the Gap

The PINN does not rely on the linear approximation ($\mathbf{H}$). Instead, it enforces the **full, non-linear governing equations** directly within its internal architecture.

- **Enforcing Non-Linearity:** In a power grid, the PINN is trained on the full AC Power Flow equations: $P_i = \sum_j V_i V_j (G_{ij} \cos\theta_{ij} + B_{ij} \sin\theta_{ij})$.
- **Higher-Order Consistency:** While a stealthy attack might satisfy a linear consistency check, it is nearly impossible for an attacker to spoof multiple variables (Voltage, Phase, Real Power, and Reactive Power) in a way that simultaneously satisfies these complex, non-linear trigonometric relationships.
- **Residual Explosion:** The PINN loss function includes a physics residual term $L_{pde} = |N[\hat{u}] - f|^2$. Even if the attack bypasses a χ^2 test, it will cause a "residual explosion" in the PINN because the fake data will violate the underlying Partial Differential Equations (PDEs).

3. Detecting "Impossible" Trajectories

Beyond static snapshots, PINNs are particularly effective because they model **system dynamics** over time.

1. **Causal Flow:** PINNs learn the "physics of causality." For example, if an attacker changes a sensor reading but the EM side-channel emission (monitored via SCA) does not show the corresponding electrical work, the PINN flags the state as physically impossible.
2. **State Persistence:** Attacks often require sudden or inconsistent changes in state variables. A PINN trained on time-dependent PDEs identifies these as violations of the system's "temporal physics".

Summary: Linear vs. PINN Defense

Feature	Traditional χ^2 Test	PINN-Based Defense
Model Type	Linear Approximation ($\mathbf{H}$)	Full Non-Linear Equations ($N[u]$)
Detection Method	Statistical distance from mean	Violation of physical laws (Residuals)
Stealth Resistance	Vulnerable to $\mathbf{a} = \mathbf{H}\mathbf{c}$	Detects attacks that bypass χ^2
Result	Attack remains "hidden"	**Residual Explosion** triggers alert

By moving the "ground truth" from a statistical average to a fundamental law of physics (like Kirchhoff's or Maxwell's equations), the PINN forces the attacker to solve a nearly impossible mathematical problem—spoofing the entire physical universe of the system—to remain undetected.

Okay suppose am trying to build a PINN for server netowork handling how to do that

To adapt the PINN pipeline for **Server Network Handling**, you move from the "Pipe Physics" of a power grid or water tank to the **"Fluid Dynamics of Data."** In high-traffic servers, packets behave like a continuous fluid, and their flow can be modeled using differential equations that represent queueing and congestion.

Here is the step-by-step pipeline for building a **PINN-based Network IDS**:

1. The Physics Oracle (Traffic Flow Equations)

Instead of PyPSA, you use **Fluid Flow Approximation** or **Queueing Theory**. The governing equation for a server's buffer/queue length (Q) usually looks like this:

$$\frac{dQ}{dt} = \lambda(t) - \mu(t) \cdot I(Q > 0)$$

- $\lambda(t)$: The incoming packet rate (Arrivals).
- $\mu(t)$: The service rate (how fast the CPU/NIC processes packets).
- $Q(t)$: The number of packets in the buffer.

This equation acts as your "Physical Law." If the arrival rate is high but the queue isn't growing, an attacker might be "dropping" or "diverting" packets stealthily.

---

2. Data Ingestion (Zeek to Semantic State)

Your **Zeek** scripts must extract the "Fluid" variables:

- **Arrival Rate:** `conn.log` (packets per second).
- **Queue Depth:** Linux `netstat` or `ss` metrics (backlog size).
- **Processing Time:** Delta between packet arrival and response.

---

3. Training the PINN (DeepXDE Implementation)

You define the **Traffic Continuity Equation** as your physics loss.

Python

```python
def network_physics(x, y):
    # x: time, y: predicted queue length Q
    Q = y[:, 0:1]
    arrival_rate = get_zeek_lambda(x) # From your logs
    service_rate = get_server_mu(x)   # Constant or measured

    dQ_dt = dde.grad.jacobian(y, x, i=0, j=0)

    # Residual: dQ/dt - (Inflow - Outflow)
    return dQ_dt - (arrival_rate - service_rate)
```

Training Data: Use "Golden Traces" of your server under normal heavy load (Stress Test data) to teach the PINN the "ideal" congestion behavior of your specific hardware.

---

4. Detecting Stealthy "Low-and-Slow" Attacks

Standard IDS look for spikes. A PINN looks for **Physical Inconsistency**.

- **The Attack:** A **Shrew Attack** or **LDoS (Low-rate DoS)** sends bursts that are too short for standard filters but cause "queue synchronization" issues.
- **The PINN Detection:** The PINN calculates the **Residual**. It sees that based on the arrival packets captured by Zeek, the server's CPU *should* be at 90% load. If the CPU reports 40%, the PINN residual spikes.
- **Confirmation:** You check the 20-packet window. If the residual is consistently high, you trigger the alert.

---

5. Deployment Architecture

- **Buffer:** Accumulate 20-50 packets of traffic metadata.
- **Inference:** Feed the (t, λ, μ) into the PINN.
- **Decision:** $||\text{Observed } Q - \text{Predicted } Q||^2 > \chi^2$.
- **Response:** Use **Wazuh** to trigger an IP tables block or a rate-limit on the offending traffic flow.

Why this is better for Servers

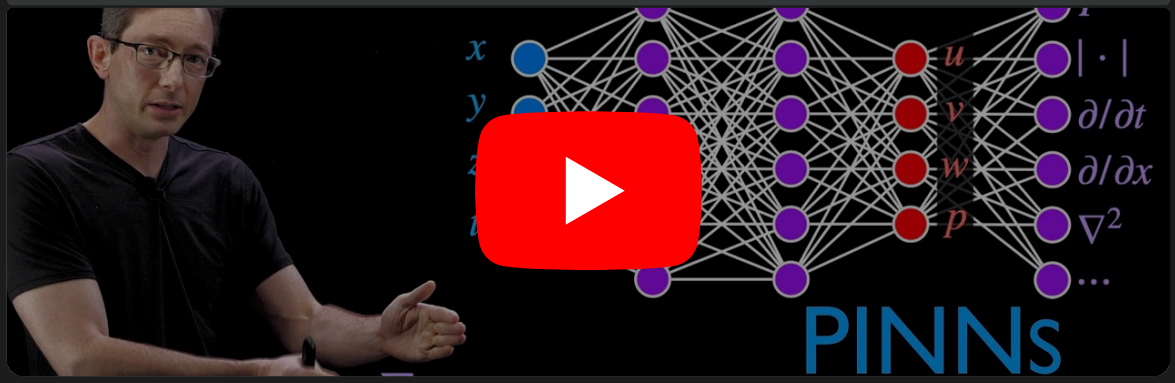
Traditional server monitoring relies on static thresholds (e.g., "Alert if CPU > 90%"). A PINN is **Context-Aware**: it knows that 90% CPU is *fine* if 1 million packets just arrived, but 90% CPU is *an attack* if only 10 packets arrived.

Physics Informed Neural Networks (PINNs) [Physics Informed Machine Learning]

This video explains the central concept of PINNs and how to add a PDE (like the traffic flow equations) into the loss function, which is the exact step needed to move your server IDS from data-driven to physics-informed.

Physics Informed Neural Networks (PINNs) [Physics Informed Machine Learning] - YouTube

Steve Brunton · 171K views



Teach me as a senoir ai developer how to build a pinn model for nyther named dvalinn that work is to prdict the values for a server all operations like network traffic and other cybersecurioty things

As the Senior AI Lead at Nyther Research, I will guide you through the architectural design and implementation of **Dvalinn**—your custom Physics-Informed Neural Network (PINN) for server infrastructure security.

Building a PINN for a server environment (where "physics" is replaced by **Queueing Theory** and **Resource Conservation Laws**) requires a shift from standard pattern recognition to "State-Space Consistency."

---

1. The "Laws of the Server" (Governing Equations)

In a server, the "physics" involves the flow of data packets and CPU cycles. We model Dvalinn using **Fluid Flow Approximations**.

The Conservation of Packets:
The rate of change of the server's buffer/queue (Q) must equal the arrival rate (λ) minus the service rate (μ).

$$\frac{dQ}{dt} = \lambda(t) - \mu(t) \cdot I(Q > 0)$$

The Conservation of Energy/Power:
The CPU power consumption (P) is generally proportional to the square of the voltage (V) and the clock frequency (f).

$$P \propto C \cdot V^2 \cdot f$$

If Zeek reports a high arrival rate (λ) but the CPU frequency (f) remains idle, Dvalinn will flag a **Physical Inconsistency**.

---

2. Dvalinn's Architecture

You will build Dvalinn using the **DeepXDE** framework. Its architecture consists of a Deep Feed-Forward Network that takes t (time) as input and outputs the system state.

The PINN Setup (Code Skeleton)

Python

```python
import deepxde as dde
import numpy as np

# 1. Define the Domain (Time window of 60 seconds)
geom = dde.geometry.TimeDomain(0, 60)

# 2. Define the "Server Physics" Residual
def server_physics(x, y):
    # y[0] = Queue Depth (Q), y[1] = CPU Load (L)
    Q = y[:, 0:1]
    L = y[:, 1:2]

    # Extract derivatives
    dQ_dt = dde.grad.jacobian(y, x, i=0, j=0)

    # Map arrival/service rates from Zeek logs
    lambda_t = get_zeek_traffic_rate(x)
    mu_t = get_processing_capacity(L)

    # The Physics Residual: dQ/dt - (Inflow - Outflow)
    return dQ_dt - (lambda_t - mu_t)

# 3. Integrate Zeek Data (Data Loss)
# 'observed_data' comes from your Zeek logs (conn.log, cpu_monitor.log)
data = dde.data.TimePDE(geom, server_physics, [ic_condition], num_domain=400, anchors=observed_data)
```

---

3. The Nyther "Stealth-Defense" Pipeline

Dvalinn doesn't just look for "bad packets"; it looks for **Physical Lying**.

1. **Ingestion: Zeek** streams network metrics (Arrival rates) into a Redis buffer.
2. **Side-Channel Fusion:** Dvalinn also takes inputs from **EM/Power probes** (as discussed in your Side-Channel research documents) to verify if the CPU is actually working as hard as the logs claim.
3. **The Windowed Residual:** As you designed, Dvalinn processes a **20-packet sliding window**.
4. **Anomaly Scoring:**
   - If $||\text{Observed Traffic} - \text{Dvalinn Prediction}||^2 > \chi^2$, Dvalinn detects that the server state is physically impossible.

---

4. Implementation Steps for the Nyther Team

Phase	Task	Senior Lead Advice
I: Data Generation	Stress test your servers.	Capture "Physics-Perfect" data during high-load but "clean" operations to use as your training baseline.
II: Noise Injection	Corrupt training data.	Add **Gaussian Noise** to the training set. This ensures Dvalinn doesn't alert on normal network jitter.
III: PINN Training	Train in **DeepXDE**.	Focus on the **Loss Weight (λ)**. If the physics loss is too low, the model becomes a standard (vulnerable) ANN. If too high, it ignores the real data.
IV: Deployment	Bridge to **Wazuh**.	Output the PINN's physical residual $r(t)$ as a custom field in Wazuh. Use it to trigger automated firewall rules.

5. Why Dvalinn is "State-of-the-Art"

Because Dvalinn is "Physics-Informed," it is immune to **Stealthy Low-and-Slow attacks**. An attacker can hide a packet burst to avoid a simple threshold, but they cannot hide the fact that the server's **Entropy** or **Thermal signature** changed in a way that contradicts the "Packet Continuity" law.

At Nyther Research, Dvalinn becomes the "Source of Truth" that verifies if the software logs are telling the truth about the hardware's reality.

Shall we start by coding the Zeek-to-DeepXDE data formatter for Dvalinn?